

1. Implement a simple linear regression algorithm to predict a continuous target variable based on a given

dataset

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

# Generate a sample dataset
np.random.seed(42)
X = 2 * np.random.rand(100, 1) # Feature (independent variable)
y = 4 + 3 * X + np.random.randn(100, 1) # Target variable with some noise

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Function to compute the coefficients for simple linear regression
def compute_coefficients(X, y):
    n = len(X)
    X_mean = np.mean(X)
    y_mean = np.mean(y)

    # Calculate slope (m) and intercept (b)
    numerator = np.sum((X - X_mean) * (y - y_mean))
    denominator = np.sum((X - X_mean)**2)
    m = numerator / denominator
    b = y_mean - m * X_mean
    return m, b

# Compute the coefficients for the training data
m, b = compute_coefficients(X_train, y_train)

# Function to make predictions
def predict(X, m, b):
    return m * X + b

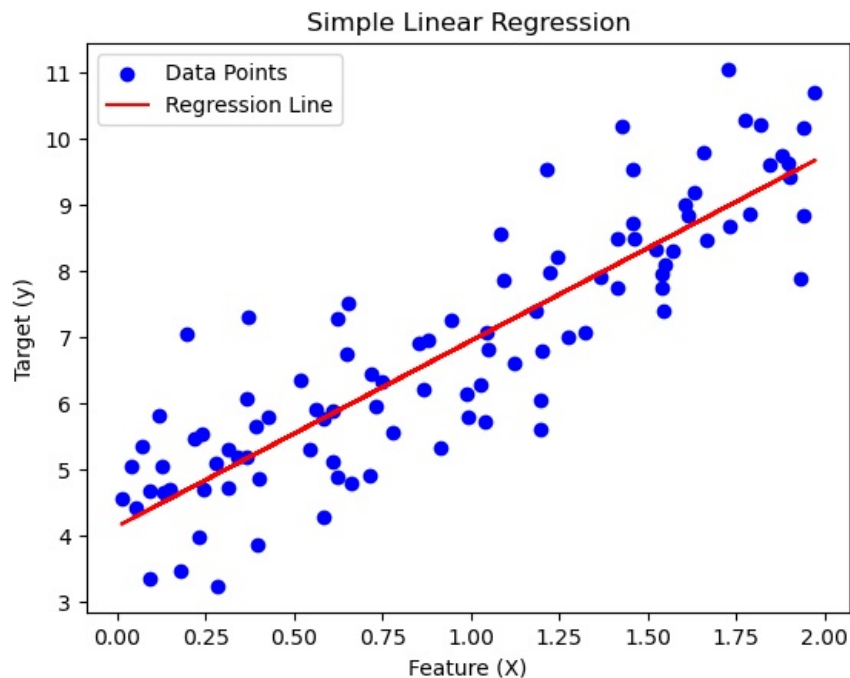
# Predict on the test data
y_pred = predict(X_test, m, b)

# Visualize the results
plt.scatter(X, y, color='blue', label='Data Points')
plt.plot(X, predict(X, m, b), color='red', label='Regression Line')
plt.xlabel("Feature (X)")
plt.ylabel("Target (y)")
plt.title("Simple Linear Regression")
plt.legend()
plt.show()

# Print the model coefficients
print(f"Slope (m): {m:.2f}")
print(f"Intercept (b): {b:.2f}")

# Evaluate the model
def mean_squared_error(y_true, y_pred):
    return np.mean((y_true - y_pred)**2)

mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error: {mse:.2f}")
```



Slope (m): 2.80
 Intercept (b): 4.14
 Mean Squared Error: 0.65

2

Develop a program to implement a Support Vector Machine for binary classification. Use a sample dataset and visualize the decision boundary.

```
In [2]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split

# Step 1: Generate a sample dataset
X, y = make_blobs(n_samples=200, centers=2, random_state=42, cluster_std=1.5)
y = y.astype(np.int_) # Binary labels (0 and 1)

# Step 2: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 3: Train the SVM model
model = SVC(kernel='linear') # Linear kernel for SVM
model.fit(X_train, y_train)

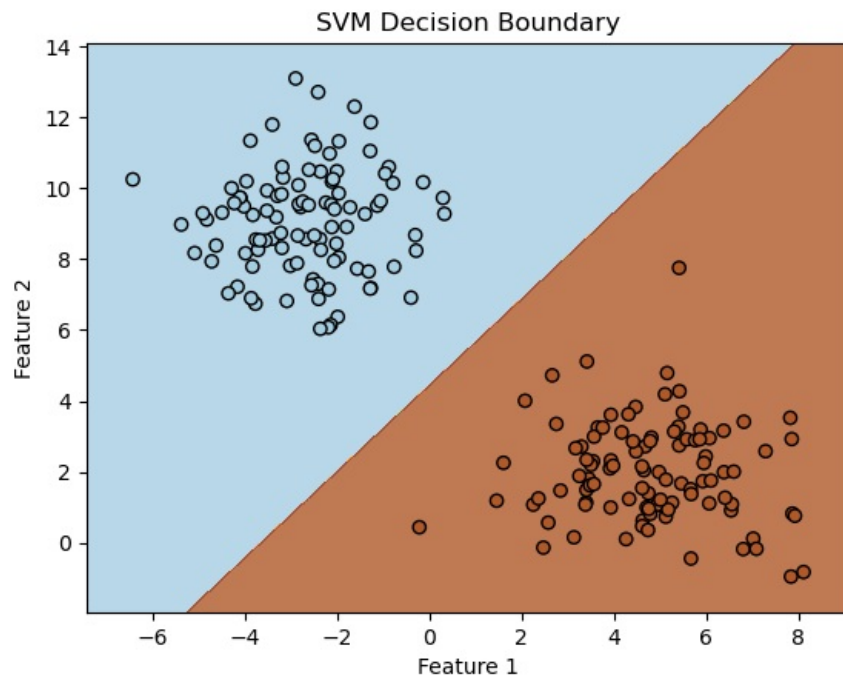
# Step 4: Visualize the decision boundary
def plot_decision_boundary(X, y, model):
    # Create a mesh grid
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01),
                          np.arange(y_min, y_max, 0.01))

    # Predict the class for each point in the mesh
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # Plot the decision boundary
    plt.contourf(xx, yy, Z, alpha=0.8, cmap=plt.cm.Paired)
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap=plt.cm.Paired)
    plt.title("SVM Decision Boundary")
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    plt.show()

# Call the function to plot
plot_decision_boundary(X, y, model)

# Step 5: Evaluate the model
accuracy = model.score(X_test, y_test)
print(f"Model Accuracy: {accuracy * 100:.2f}%")
```



Model Accuracy: 100.00%

Develop a simple case-based reasoning system that stores instances of past cases. Implement a retrieval method to find the most similar cases and make predictions based on them.

```
In [3]: import numpy as np
from scipy.spatial.distance import euclidean

class CaseBasedReasoning:
    def __init__(self):
        self.cases = [] # Store past cases as tuples of (features, label)

    def add_case(self, features, label):
        """
        Add a new case to the case base.
        :param features: List of features (problem description)
        :param label: The solution or result for the case
        """
        self.cases.append((np.array(features), label))

    def retrieve(self, query, k=1):
        """
        Retrieve the k most similar cases to the query.
        :param query: List of features for the query case
        :param k: Number of similar cases to retrieve
        :return: List of the k most similar cases
        """
        distances = [(case, euclidean(query, case[0])) for case in self.cases]
        distances.sort(key=lambda x: x[1]) # Sort by distance
        return distances[:k]

    def predict(self, query, k=1):
        """
        Predict the label for the query case based on the k nearest cases.
        :param query: List of features for the query case
        :param k: Number of similar cases to use for prediction
        :return: Predicted label
        """
        similar_cases = self.retrieve(query, k)
        labels = [case[0][1] for case in similar_cases]
        # Return the most common label
        return max(set(labels), key=labels.count)

# Example Usage
if __name__ == "__main__":
    # Initialize the CBR system
    cbr = CaseBasedReasoning()

    # Add some past cases (features, label)
    cbr.add_case([2.5, 3.0], "Class A")
    cbr.add_case([1.0, 1.5], "Class B")
    cbr.add_case([3.0, 4.0], "Class A")
    cbr.add_case([0.5, 1.0], "Class B")

    # Query a new case
    query_case = [2.7, 3.5]
    print("Query Case:", query_case)

    # Retrieve similar cases
```

```

similar_cases = cbr.retrieve(query_case, k=2)
print("\nMost Similar Cases:")
for case, distance in similar_cases:
    print(f"Features: {case[0]}, Label: {case[1]}, Distance: {distance:.2f}")

# Predict the label for the query case
prediction = cbr.predict(query_case, k=2)
print("\nPredicted Label:", prediction)

```

Query Case: [2.7, 3.5]

Most Similar Cases:

Features: [2.5 3.], Label: Class A, Distance: 0.54

Features: [3. 4.], Label: Class A, Distance: 0.58

Predicted Label: Class A

1. Write a program to demonstrate the ID3 decision tree algorithm using an appropriate dataset for classification.

```

In [4]: from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# Step 1: Load a dataset (Iris dataset for demonstration)
data = load_iris()
X = data.data # Features
y = data.target # Target labels
feature_names = data.feature_names
class_names = data.target_names

# Step 2: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 3: Train a decision tree classifier using the "entropy" criterion (ID3 algorithm)
id3_tree = DecisionTreeClassifier(criterion="entropy", random_state=42)
id3_tree.fit(X_train, y_train)

# Step 4: Evaluate the model
y_pred = id3_tree.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy * 100:.2f}%")

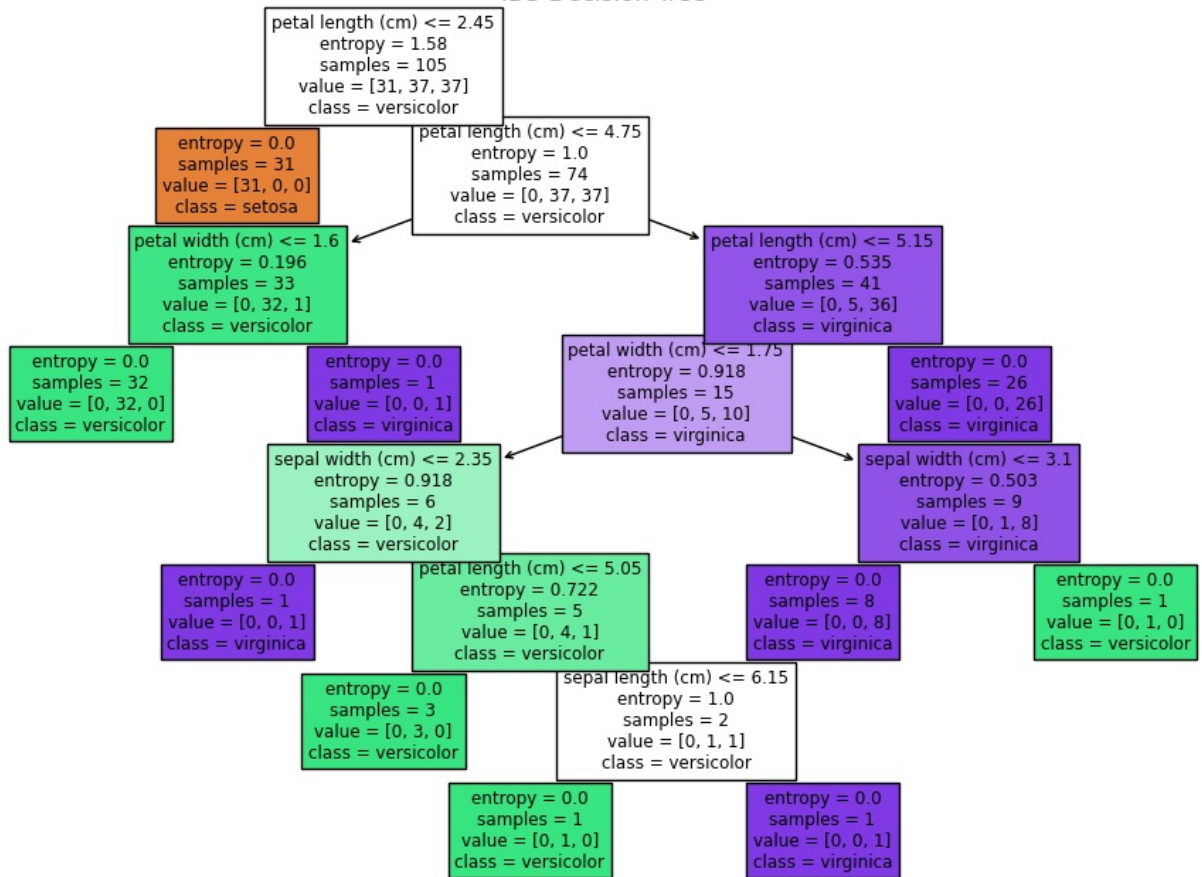
# Step 5: Visualize the decision tree
plt.figure(figsize=(12, 8))
plot_tree(id3_tree, feature_names=feature_names, class_names=class_names, filled=True)
plt.title("ID3 Decision Tree")
plt.show()

# Step 6: Display the rules of the decision tree
tree_rules = export_text(id3_tree, feature_names=feature_names)
print("\nDecision Tree Rules:\n")
print(tree_rules)

```

Model Accuracy: 97.78%

ID3 Decision Tree



Decision Tree Rules:

```

|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal length (cm) <= 4.75
|   |   |--- petal width (cm) <= 1.60
|   |   |   |--- class: 1
|   |   |--- petal width (cm) > 1.60
|   |   |   |--- class: 2
|   |--- petal length (cm) > 4.75
|   |   |--- petal length (cm) <= 5.15
|   |   |   |--- petal width (cm) <= 1.75
|   |   |   |   |--- sepal width (cm) <= 2.35
|   |   |   |   |   |--- class: 2
|   |   |   |   |--- sepal width (cm) > 2.35
|   |   |   |   |   |--- petal length (cm) <= 5.05
|   |   |   |   |   |   |--- class: 1
|   |   |   |   |   |--- petal length (cm) > 5.05
|   |   |   |   |       |--- sepal length (cm) <= 6.15
|   |   |   |   |       |   |--- class: 1
|   |   |   |   |       |--- sepal length (cm) > 6.15
|   |   |   |   |       |   |--- class: 2
|   |   |   |--- petal width (cm) > 1.75
|   |   |   |   |--- sepal width (cm) <= 3.10
|   |   |   |   |   |--- class: 2
|   |   |   |   |--- sepal width (cm) > 3.10
|   |   |   |   |   |--- class: 1
|   |--- petal length (cm) > 5.15
|   |   |--- class: 2

```

1. Build an Artificial Neural Network by implementing the Backpropagation algorithm and test it with suitable datasets.

In [5]: `import numpy as np`

```

# Step 1: Define the sigmoid activation function and its derivative
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# Step 2: Define the neural network architecture
class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size, learning_rate=0.1):
        # Initialize weights and biases
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size
        self.learning_rate = learning_rate

        # Randomly initialize weights and biases
        self.weights_input_hidden = np.random.rand(input_size, hidden_size)
        self.bias_hidden = np.random.rand(hidden_size)
        self.weights_hidden_output = np.random.rand(hidden_size, output_size)
        self.bias_output = np.random.rand(output_size)

    def forward(self, X):
        # Forward propagation
        self.input_layer = X
        self.hidden_layer_input = np.dot(X, self.weights_input_hidden) + self.bias_hidden
        self.hidden_layer_output = sigmoid(self.hidden_layer_input)
        self.output_layer_input = np.dot(self.hidden_layer_output, self.weights_hidden_output) + self.bias_output
        self.output_layer_output = sigmoid(self.output_layer_input)
        return self.output_layer_output

    def backward(self, X, y, output):
        # Backward propagation
        # Calculate output layer error and delta
        output_error = y - output
        output_delta = output_error * sigmoid_derivative(output)

        # Calculate hidden layer error and delta
        hidden_error = np.dot(output_delta, self.weights_hidden_output.T)
        hidden_delta = hidden_error * sigmoid_derivative(self.hidden_layer_output)

        # Update weights and biases
        self.weights_hidden_output += np.dot(self.hidden_layer_output.T, output_delta) * self.learning_rate
        self.bias_output += np.sum(output_delta, axis=0) * self.learning_rate
        self.weights_input_hidden += np.dot(X.T, hidden_delta) * self.learning_rate
        self.bias_hidden += np.sum(hidden_delta, axis=0) * self.learning_rate

    def train(self, X, y, epochs):
        for epoch in range(epochs):
            # Perform forward and backward propagation

```

```

        output = self.forward(X)
        self.backward(X, y, output)
        # Calculate and print loss
        loss = np.mean((y - output) ** 2)
        if (epoch + 1) % 1000 == 0:
            print(f"Epoch {epoch + 1}/{epochs}, Loss: {loss:.4f}")

# Step 3: Train the neural network on a dataset (e.g., XOR problem)
if __name__ == "__main__":
    # XOR dataset
    X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]]) # Input features
    y = np.array([[0], [1], [1], [0]]) # Target labels

    # Define the neural network
    input_size = 2
    hidden_size = 4
    output_size = 1
    learning_rate = 0.1
    epochs = 10000

    nn = NeuralNetwork(input_size, hidden_size, output_size, learning_rate)

    # Train the neural network
    print("Training the Neural Network...")
    nn.train(X, y, epochs)

    # Test the neural network
    print("\nTesting the Neural Network...")
    for i, test_input in enumerate(X):
        predicted_output = nn.forward(test_input)
        print(f"Input: {test_input}, Predicted Output: {predicted_output[0]:.4f}, Expected: {y[i][0]}")

```

```

Training the Neural Network...
Epoch 1000/10000, Loss: 0.2475
Epoch 2000/10000, Loss: 0.2313
Epoch 3000/10000, Loss: 0.1764
Epoch 4000/10000, Loss: 0.0873
Epoch 5000/10000, Loss: 0.0247
Epoch 6000/10000, Loss: 0.0111
Epoch 7000/10000, Loss: 0.0066
Epoch 8000/10000, Loss: 0.0046
Epoch 9000/10000, Loss: 0.0035
Epoch 10000/10000, Loss: 0.0027

```

```

Testing the Neural Network...
Input: [0 0], Predicted Output: 0.0556, Expected: 0
Input: [0 1], Predicted Output: 0.9501, Expected: 1
Input: [1 0], Predicted Output: 0.9488, Expected: 1
Input: [1 1], Predicted Output: 0.0527, Expected: 0

```

1. Implement a KNN algorithm for regression tasks instead of classification. Use a small dataset, and predict continuous values based on the average of the nearest neighbors.

```

In [6]: import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt

# Step 1: Define the KNN regression algorithm
class KNNRegressor:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X, y):
        self.X_train = X
        self.y_train = y

    def predict(self, X_test):
        predictions = []
        for x in X_test:
            # Calculate distances from x to all training points
            distances = np.sqrt(np.sum((self.X_train - x) ** 2, axis=1))
            # Get the indices of the k nearest neighbors
            nearest_neighbors_indices = np.argsort(distances)[:self.k]
            # Compute the average of the nearest neighbors' target values
            nearest_neighbors_targets = self.y_train[nearest_neighbors_indices]
            predictions.append(np.mean(nearest_neighbors_targets))
        return np.array(predictions)

# Step 2: Create a small dataset for regression
# Example: Predicting house prices based on size (in square feet)
X = np.array([[600], [800], [1000], [1200], [1400], [1600], [1800], [2000]]) # Feature: Size
y = np.array([150, 200, 250, 300, 350, 400, 450, 500]) # Target: Price (in $1000s)

# Normalize the dataset for better performance
X = (X - X.min()) / (X.max() - X.min())

```

```

# Step 3: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Step 4: Train the KNN regressor
k = 3
knn_regressor = KNNRegressor(k=k)
knn_regressor.fit(X_train, y_train)

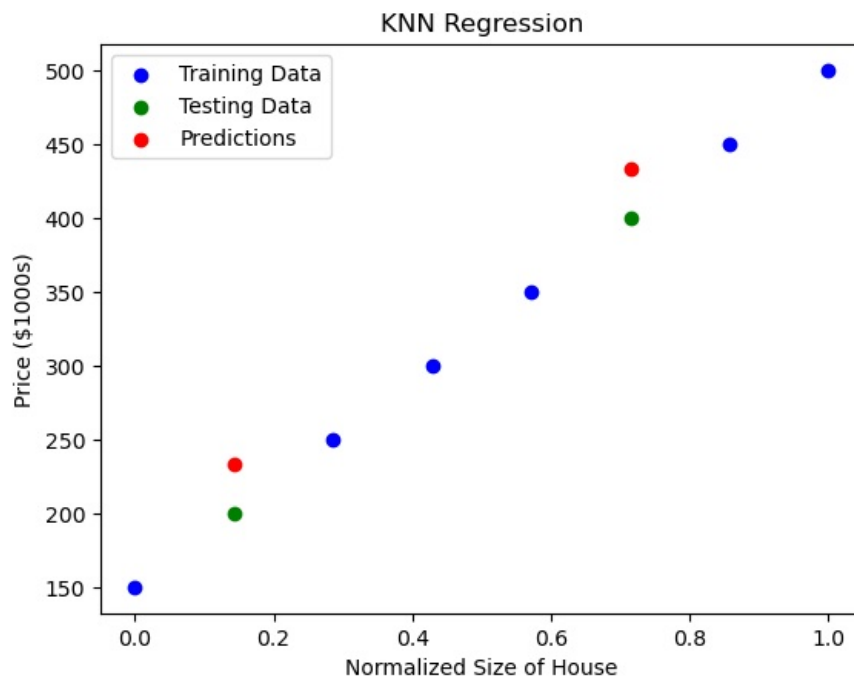
# Step 5: Predict on the test set
y_pred = knn_regressor.predict(X_test)

# Step 6: Evaluate the model
mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error: {mse:.2f}")

# Step 7: Visualize the results
plt.scatter(X_train, y_train, color='blue', label='Training Data')
plt.scatter(X_test, y_test, color='green', label='Testing Data')
plt.scatter(X_test, y_pred, color='red', label='Predictions')
plt.xlabel('Normalized Size of House')
plt.ylabel('Price ($1000s)')
plt.title('KNN Regression')
plt.legend()
plt.show()

```

Mean Squared Error: 1111.11



7. Create a program that calculates different distance metrics (Euclidean and Manhattan) between two points in a dataset. Allow the user to input two points and display the calculated distances.

```

In [8]: import numpy as np

# Step 1: Define functions to calculate distance metrics
def euclidean_distance(point1, point2):
    return np.sqrt(np.sum((point1 - point2) ** 2))

def manhattan_distance(point1, point2):
    return np.sum(np.abs(point1 - point2))

# Step 2: Define the main function
def main():
    print("Distance Metrics Calculator (Euclidean & Manhattan)\n")

    try:
        # User inputs for two points
        point1 = input("Enter the first point (comma-separated values, e.g., 2,3): ")
        point2 = input("Enter the second point (comma-separated values, e.g., 5,7): ")

        # Convert the inputs to numpy arrays
        point1 = np.array([float(x) for x in point1.split(",")])
        point2 = np.array([float(x) for x in point2.split(",")])

        # Check if the dimensions of the points match
        if point1.shape != point2.shape:
            print("Error: Points must have the same dimensions!")
            return

        # Calculate distances
        euclidean = euclidean_distance(point1, point2)

```

```

        manhattan = manhattan_distance(point1, point2)

    # Display results
    print(f"\nDistances between {point1} and {point2}:")
    print(f"Euclidean Distance: {euclidean:.4f}")
    print(f"Manhattan Distance: {manhattan:.4f}")

    except ValueError:
        print("Error: Invalid input. Please enter numerical values separated by commas.")

# Run the program
if __name__ == "__main__":
    main()

```

Distance Metrics Calculator (Euclidean & Manhattan)

Enter the first point (comma-separated values, e.g., 2,3): 4,6
 Enter the second point (comma-separated values, e.g., 5,7): 8,9

Distances between [4. 6.] and [8. 9.]:
 Euclidean Distance: 5.0000
 Manhattan Distance: 7.0000

8. Implement the k-Nearest Neighbor algorithm to classify the Iris dataset, printing both correct and incorrect predictions.

```

In [9]: import numpy as np
        from sklearn.datasets import load_iris
        from sklearn.model_selection import train_test_split
        from sklearn.metrics import accuracy_score

        # Step 1: Define the KNN algorithm
        class KNNClassifier:
            def __init__(self, k=3):
                self.k = k

            def fit(self, X_train, y_train):
                self.X_train = X_train
                self.y_train = y_train

            def predict(self, X_test):
                predictions = []
                for x in X_test:
                    # Compute distances from x to all training points
                    distances = np.sqrt(np.sum((self.X_train - x) ** 2, axis=1))
                    # Get indices of k nearest neighbors
                    nearest_neighbors_indices = np.argsort(distances)[:self.k]
                    # Find the most common class among neighbors
                    nearest_neighbors_labels = self.y_train[nearest_neighbors_indices]
                    most_common_label = np.bincount(nearest_neighbors_labels).argmax()
                    predictions.append(most_common_label)
                return np.array(predictions)

        # Step 2: Load the Iris dataset
        iris = load_iris()
        X = iris.data # Features
        y = iris.target # Labels (classes)

        # Step 3: Split the data into training and testing sets
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

        # Step 4: Train the KNN classifier
        k = 5 # Number of neighbors
        knn = KNNClassifier(k=k)
        knn.fit(X_train, y_train)

        # Step 5: Predict on the test set
        y_pred = knn.predict(X_test)

        # Step 6: Evaluate and print results
        accuracy = accuracy_score(y_test, y_pred)
        print(f"Accuracy: {accuracy * 100:.2f}%")

        # Step 7: Print correct and incorrect predictions
        print("\nCorrect Predictions:")
        for i in range(len(y_test)):
            if y_test[i] == y_pred[i]:
                print(f"Test Sample {i}: True Label = {y_test[i]}, Predicted Label = {y_pred[i]}")

        print("\nIncorrect Predictions:")
        for i in range(len(y_test)):
            if y_test[i] != y_pred[i]:
                print(f"Test Sample {i}: True Label = {y_test[i]}, Predicted Label = {y_pred[i]}")

```

Accuracy: 100.00%

Correct Predictions:

Test Sample 0: True Label = 1, Predicted Label = 1
Test Sample 1: True Label = 0, Predicted Label = 0
Test Sample 2: True Label = 2, Predicted Label = 2
Test Sample 3: True Label = 1, Predicted Label = 1
Test Sample 4: True Label = 1, Predicted Label = 1
Test Sample 5: True Label = 0, Predicted Label = 0
Test Sample 6: True Label = 1, Predicted Label = 1
Test Sample 7: True Label = 2, Predicted Label = 2
Test Sample 8: True Label = 1, Predicted Label = 1
Test Sample 9: True Label = 1, Predicted Label = 1
Test Sample 10: True Label = 2, Predicted Label = 2
Test Sample 11: True Label = 0, Predicted Label = 0
Test Sample 12: True Label = 0, Predicted Label = 0
Test Sample 13: True Label = 0, Predicted Label = 0
Test Sample 14: True Label = 0, Predicted Label = 0
Test Sample 15: True Label = 1, Predicted Label = 1
Test Sample 16: True Label = 2, Predicted Label = 2
Test Sample 17: True Label = 1, Predicted Label = 1
Test Sample 18: True Label = 1, Predicted Label = 1
Test Sample 19: True Label = 2, Predicted Label = 2
Test Sample 20: True Label = 0, Predicted Label = 0
Test Sample 21: True Label = 2, Predicted Label = 2
Test Sample 22: True Label = 0, Predicted Label = 0
Test Sample 23: True Label = 2, Predicted Label = 2
Test Sample 24: True Label = 2, Predicted Label = 2
Test Sample 25: True Label = 2, Predicted Label = 2
Test Sample 26: True Label = 2, Predicted Label = 2
Test Sample 27: True Label = 2, Predicted Label = 2
Test Sample 28: True Label = 0, Predicted Label = 0
Test Sample 29: True Label = 0, Predicted Label = 0
Test Sample 30: True Label = 0, Predicted Label = 0
Test Sample 31: True Label = 0, Predicted Label = 0
Test Sample 32: True Label = 1, Predicted Label = 1
Test Sample 33: True Label = 0, Predicted Label = 0
Test Sample 34: True Label = 0, Predicted Label = 0
Test Sample 35: True Label = 2, Predicted Label = 2
Test Sample 36: True Label = 1, Predicted Label = 1
Test Sample 37: True Label = 0, Predicted Label = 0

Incorrect Predictions:

1. Develop a program to implement the non-parametric Locally Weighted Regression algorithm, fitting data points and visualizing results.

```
In [11]: import numpy as np
import matplotlib.pyplot as plt

# Step 1: Define the kernel function (Gaussian)
def kernel(x, x_i, tau):
    return np.exp(-np.sum((x - x_i) ** 2) / (2 * tau ** 2))

# Step 2: Implement the Locally Weighted Regression algorithm
def locally_weighted_regression(X, y, x_query, tau):
    """
    Perform Locally Weighted Regression for a given query point.
    :param X: Training features (n_samples, n_features)
    :param y: Training target values (n_samples,)
    :param x_query: The query point (1, n_features)
    :param tau: Bandwidth parameter (controls smoothness)
    :return: Predicted value for the query point
    """
    X = np.hstack((np.ones((X.shape[0], 1)), X)) # Add bias term
    x_query = np.hstack([[1], x_query]) # Add bias term to query point

    # Compute weights for each training point
    weights = np.array([kernel(x_query, x_i, tau) for x_i in X])

    # Compute weighted normal equation: theta = (X^T W X)^-1 X^T W y
    W = np.diag(weights) # Weight matrix
    theta = np.linalg.inv(X.T @ W @ X) @ X.T @ W @ y
    return x_query @ theta # Predicted value

# Step 3: Visualize the results with a sample dataset
def main():
    # Generate synthetic data (y = sin(x) + noise)
    np.random.seed(42)
    X = np.linspace(0, 10, 50).reshape(-1, 1)
    y = np.sin(X).ravel() + np.random.normal(scale=0.1, size=X.shape[0])

    # Query points for prediction
    X_query = np.linspace(0, 10, 100).reshape(-1, 1)
    tau = 0.5 # Bandwidth parameter (adjustable)
```

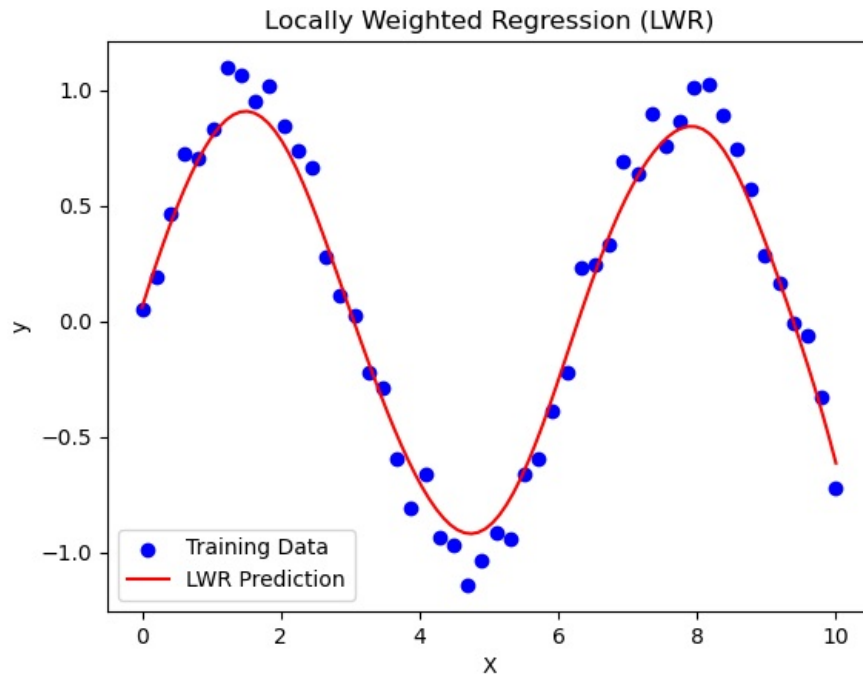
```

# Predict values for query points
y_pred = np.array([locally_weighted_regression(X, y, x_query, tau) for x_query in X_query])

# Plot the results
plt.scatter(X, y, color="blue", label="Training Data")
plt.plot(X_query, y_pred, color="red", label="LWR Prediction")
plt.xlabel("X")
plt.ylabel("y")
plt.title("Locally Weighted Regression (LWR)")
plt.legend()
plt.show()

if __name__ == "__main__":
    main()

```



#

1. Implement a Q-learning algorithm to navigate a simple grid environment, defining the reward structure and analyzing agent performance.

```

In [12]: import numpy as np
import random

# Step 1: Define the grid environment
class GridEnvironment:
    def __init__(self, size=(4, 4), start=(0, 0), goal=(3, 3)):
        self.size = size
        self.start = start
        self.goal = goal
        self.state = start
        self.actions = ['up', 'down', 'left', 'right']

    def reset(self):
        self.state = self.start
        return self.state

    def step(self, action):
        x, y = self.state
        if action == 'up' and x > 0:
            x -= 1
        elif action == 'down' and x < self.size[0] - 1:
            x += 1
        elif action == 'left' and y > 0:
            y -= 1
        elif action == 'right' and y < self.size[1] - 1:
            y += 1

        self.state = (x, y)
        reward = 1 if self.state == self.goal else -0.1 # Reward structure
        done = self.state == self.goal
        return self.state, reward, done

# Step 2: Define the Q-learning agent
class QLearningAgent:
    def __init__(self, env, learning_rate=0.1, discount_factor=0.9, epsilon=0.1):
        self.env = env
        self.lr = learning_rate
        self.gamma = discount_factor

```

```

self.epsilon = epsilon
self.q_table = {}
self._initialize_q_table()

def _initialize_q_table(self):
    for x in range(self.env.size[0]):
        for y in range(self.env.size[1]):
            self.q_table[(x, y)] = {action: 0 for action in self.env.actions}

def choose_action(self, state):
    if random.random() < self.epsilon: # Explore
        return random.choice(self.env.actions)
    else: # Exploit
        return max(self.q_table[state], key=self.q_table[state].get)

def update_q_value(self, state, action, reward, next_state):
    best_next_action = max(self.q_table[next_state], key=self.q_table[next_state].get)
    td_target = reward + self.gamma * self.q_table[next_state][best_next_action]
    td_error = td_target - self.q_table[state][action]
    self.q_table[state][action] += self.lr * td_error

def train(self, episodes):
    for episode in range(episodes):
        state = self.env.reset()
        total_reward = 0
        done = False

        while not done:
            action = self.choose_action(state)
            next_state, reward, done = self.env.step(action)
            self.update_q_value(state, action, reward, next_state)
            state = next_state
            total_reward += reward

        print(f"Episode {episode + 1}: Total Reward = {total_reward:.2f}")

def test(self):
    state = self.env.reset()
    path = [state]
    total_reward = 0
    done = False

    while not done:
        action = max(self.q_table[state], key=self.q_table[state].get)
        state, reward, done = self.env.step(action)
        path.append(state)
        total_reward += reward

    print(f"Test Run: Path = {path}, Total Reward = {total_reward:.2f}")

# Step 3: Main Program
if __name__ == "__main__":
    env = GridEnvironment(size=(4, 4), start=(0, 0), goal=(3, 3))
    agent = QLearningAgent(env, learning_rate=0.1, discount_factor=0.9, epsilon=0.2)

    print("Training the agent...\n")
    agent.train(episodes=500)

    print("\nTesting the agent...\n")
    agent.test()

```

Training the agent...

```

Episode 1: Total Reward = -3.10
Episode 2: Total Reward = -3.30
Episode 3: Total Reward = -0.10
Episode 4: Total Reward = -0.80
Episode 5: Total Reward = -2.20
Episode 6: Total Reward = -1.50
Episode 7: Total Reward = -1.60
Episode 8: Total Reward = -0.80
Episode 9: Total Reward = -0.10
Episode 10: Total Reward = 0.10
Episode 11: Total Reward = 0.10
Episode 12: Total Reward = -0.40
Episode 13: Total Reward = -0.40
Episode 14: Total Reward = -0.20
Episode 15: Total Reward = 0.00
Episode 16: Total Reward = 0.40
Episode 17: Total Reward = 0.20
Episode 18: Total Reward = -0.10
Episode 19: Total Reward = -0.60
Episode 20: Total Reward = -0.20
Episode 21: Total Reward = -1.00
Episode 22: Total Reward = 0.50
Episode 23: Total Reward = 0.50
Episode 24: Total Reward = 0.30
Episode 25: Total Reward = 0.10
Episode 26: Total Reward = 0.50

```

Episode 27: Total Reward = 0.00
Episode 28: Total Reward = 0.00
Episode 29: Total Reward = 0.40
Episode 30: Total Reward = 0.00
Episode 31: Total Reward = 0.30
Episode 32: Total Reward = 0.50
Episode 33: Total Reward = 0.10
Episode 34: Total Reward = 0.10
Episode 35: Total Reward = 0.40
Episode 36: Total Reward = 0.10
Episode 37: Total Reward = 0.50
Episode 38: Total Reward = 0.00
Episode 39: Total Reward = 0.50
Episode 40: Total Reward = 0.50
Episode 41: Total Reward = 0.40
Episode 42: Total Reward = 0.50
Episode 43: Total Reward = 0.30
Episode 44: Total Reward = 0.50
Episode 45: Total Reward = 0.50
Episode 46: Total Reward = 0.50
Episode 47: Total Reward = 0.30
Episode 48: Total Reward = -0.50
Episode 49: Total Reward = 0.50
Episode 50: Total Reward = 0.50
Episode 51: Total Reward = 0.00
Episode 52: Total Reward = 0.10
Episode 53: Total Reward = 0.50
Episode 54: Total Reward = 0.30
Episode 55: Total Reward = 0.40
Episode 56: Total Reward = 0.40
Episode 57: Total Reward = 0.20
Episode 58: Total Reward = 0.00
Episode 59: Total Reward = 0.50
Episode 60: Total Reward = 0.50
Episode 61: Total Reward = 0.50
Episode 62: Total Reward = 0.50
Episode 63: Total Reward = 0.50
Episode 64: Total Reward = 0.50
Episode 65: Total Reward = 0.50
Episode 66: Total Reward = 0.30
Episode 67: Total Reward = 0.50
Episode 68: Total Reward = 0.50
Episode 69: Total Reward = 0.50
Episode 70: Total Reward = 0.30
Episode 71: Total Reward = 0.40
Episode 72: Total Reward = 0.50
Episode 73: Total Reward = 0.50
Episode 74: Total Reward = 0.50
Episode 75: Total Reward = 0.50
Episode 76: Total Reward = 0.50
Episode 77: Total Reward = 0.40
Episode 78: Total Reward = 0.10
Episode 79: Total Reward = 0.50
Episode 80: Total Reward = 0.50
Episode 81: Total Reward = 0.40
Episode 82: Total Reward = -0.70
Episode 83: Total Reward = 0.50
Episode 84: Total Reward = 0.20
Episode 85: Total Reward = -0.20
Episode 86: Total Reward = 0.50
Episode 87: Total Reward = 0.10
Episode 88: Total Reward = 0.30
Episode 89: Total Reward = 0.10
Episode 90: Total Reward = 0.30
Episode 91: Total Reward = 0.50
Episode 92: Total Reward = 0.30
Episode 93: Total Reward = 0.10
Episode 94: Total Reward = 0.50
Episode 95: Total Reward = 0.40
Episode 96: Total Reward = 0.30
Episode 97: Total Reward = 0.20
Episode 98: Total Reward = 0.50
Episode 99: Total Reward = 0.20
Episode 100: Total Reward = 0.30
Episode 101: Total Reward = 0.50
Episode 102: Total Reward = 0.20
Episode 103: Total Reward = 0.30
Episode 104: Total Reward = 0.50
Episode 105: Total Reward = 0.50
Episode 106: Total Reward = 0.30
Episode 107: Total Reward = 0.30
Episode 108: Total Reward = 0.10
Episode 109: Total Reward = 0.20
Episode 110: Total Reward = 0.30
Episode 111: Total Reward = 0.30
Episode 112: Total Reward = -0.70
Episode 113: Total Reward = 0.50
Episode 114: Total Reward = 0.50
Episode 115: Total Reward = 0.50

Episode 116: Total Reward = 0.20
Episode 117: Total Reward = 0.50
Episode 118: Total Reward = 0.30
Episode 119: Total Reward = 0.10
Episode 120: Total Reward = 0.50
Episode 121: Total Reward = 0.20
Episode 122: Total Reward = 0.50
Episode 123: Total Reward = 0.50
Episode 124: Total Reward = 0.40
Episode 125: Total Reward = 0.30
Episode 126: Total Reward = -0.10
Episode 127: Total Reward = 0.50
Episode 128: Total Reward = -0.10
Episode 129: Total Reward = 0.50
Episode 130: Total Reward = 0.30
Episode 131: Total Reward = 0.50
Episode 132: Total Reward = 0.40
Episode 133: Total Reward = 0.50
Episode 134: Total Reward = 0.50
Episode 135: Total Reward = 0.30
Episode 136: Total Reward = 0.30
Episode 137: Total Reward = 0.50
Episode 138: Total Reward = 0.50
Episode 139: Total Reward = 0.50
Episode 140: Total Reward = 0.30
Episode 141: Total Reward = 0.50
Episode 142: Total Reward = 0.50
Episode 143: Total Reward = 0.50
Episode 144: Total Reward = 0.30
Episode 145: Total Reward = 0.20
Episode 146: Total Reward = 0.50
Episode 147: Total Reward = 0.50
Episode 148: Total Reward = 0.50
Episode 149: Total Reward = 0.50
Episode 150: Total Reward = 0.40
Episode 151: Total Reward = -0.10
Episode 152: Total Reward = 0.50
Episode 153: Total Reward = 0.50
Episode 154: Total Reward = 0.50
Episode 155: Total Reward = 0.50
Episode 156: Total Reward = 0.40
Episode 157: Total Reward = 0.50
Episode 158: Total Reward = 0.50
Episode 159: Total Reward = 0.50
Episode 160: Total Reward = 0.50
Episode 161: Total Reward = 0.50
Episode 162: Total Reward = 0.50
Episode 163: Total Reward = 0.50
Episode 164: Total Reward = 0.30
Episode 165: Total Reward = 0.20
Episode 166: Total Reward = 0.50
Episode 167: Total Reward = 0.50
Episode 168: Total Reward = 0.50
Episode 169: Total Reward = 0.50
Episode 170: Total Reward = 0.50
Episode 171: Total Reward = 0.50
Episode 172: Total Reward = 0.00
Episode 173: Total Reward = 0.40
Episode 174: Total Reward = 0.30
Episode 175: Total Reward = 0.50
Episode 176: Total Reward = 0.50
Episode 177: Total Reward = 0.30
Episode 178: Total Reward = 0.50
Episode 179: Total Reward = 0.40
Episode 180: Total Reward = 0.50
Episode 181: Total Reward = 0.30
Episode 182: Total Reward = 0.50
Episode 183: Total Reward = 0.50
Episode 184: Total Reward = 0.30
Episode 185: Total Reward = 0.30
Episode 186: Total Reward = 0.50
Episode 187: Total Reward = 0.40
Episode 188: Total Reward = 0.50
Episode 189: Total Reward = 0.50
Episode 190: Total Reward = 0.50
Episode 191: Total Reward = 0.00
Episode 192: Total Reward = 0.50
Episode 193: Total Reward = 0.50
Episode 194: Total Reward = 0.30
Episode 195: Total Reward = 0.50
Episode 196: Total Reward = 0.30
Episode 197: Total Reward = 0.50
Episode 198: Total Reward = 0.30
Episode 199: Total Reward = 0.30
Episode 200: Total Reward = 0.50
Episode 201: Total Reward = 0.20
Episode 202: Total Reward = 0.30
Episode 203: Total Reward = 0.10
Episode 204: Total Reward = 0.50

Episode 205: Total Reward = 0.30
Episode 206: Total Reward = 0.50
Episode 207: Total Reward = 0.50
Episode 208: Total Reward = 0.50
Episode 209: Total Reward = 0.50
Episode 210: Total Reward = 0.10
Episode 211: Total Reward = 0.50
Episode 212: Total Reward = 0.50
Episode 213: Total Reward = 0.30
Episode 214: Total Reward = 0.50
Episode 215: Total Reward = 0.50
Episode 216: Total Reward = 0.30
Episode 217: Total Reward = 0.30
Episode 218: Total Reward = 0.50
Episode 219: Total Reward = 0.50
Episode 220: Total Reward = -0.10
Episode 221: Total Reward = 0.40
Episode 222: Total Reward = 0.50
Episode 223: Total Reward = 0.50
Episode 224: Total Reward = 0.10
Episode 225: Total Reward = 0.10
Episode 226: Total Reward = 0.40
Episode 227: Total Reward = 0.40
Episode 228: Total Reward = 0.40
Episode 229: Total Reward = 0.10
Episode 230: Total Reward = 0.50
Episode 231: Total Reward = 0.50
Episode 232: Total Reward = 0.10
Episode 233: Total Reward = 0.10
Episode 234: Total Reward = 0.50
Episode 235: Total Reward = 0.50
Episode 236: Total Reward = 0.20
Episode 237: Total Reward = 0.50
Episode 238: Total Reward = 0.50
Episode 239: Total Reward = 0.30
Episode 240: Total Reward = 0.10
Episode 241: Total Reward = 0.50
Episode 242: Total Reward = 0.50
Episode 243: Total Reward = 0.50
Episode 244: Total Reward = 0.30
Episode 245: Total Reward = 0.40
Episode 246: Total Reward = 0.30
Episode 247: Total Reward = 0.30
Episode 248: Total Reward = 0.50
Episode 249: Total Reward = 0.50
Episode 250: Total Reward = 0.50
Episode 251: Total Reward = 0.20
Episode 252: Total Reward = 0.50
Episode 253: Total Reward = 0.50
Episode 254: Total Reward = 0.00
Episode 255: Total Reward = 0.50
Episode 256: Total Reward = 0.50
Episode 257: Total Reward = 0.30
Episode 258: Total Reward = 0.30
Episode 259: Total Reward = 0.50
Episode 260: Total Reward = 0.50
Episode 261: Total Reward = 0.30
Episode 262: Total Reward = 0.30
Episode 263: Total Reward = 0.20
Episode 264: Total Reward = 0.50
Episode 265: Total Reward = 0.30
Episode 266: Total Reward = 0.30
Episode 267: Total Reward = 0.30
Episode 268: Total Reward = 0.50
Episode 269: Total Reward = 0.50
Episode 270: Total Reward = 0.30
Episode 271: Total Reward = 0.30
Episode 272: Total Reward = 0.30
Episode 273: Total Reward = 0.50
Episode 274: Total Reward = 0.50
Episode 275: Total Reward = 0.30
Episode 276: Total Reward = 0.20
Episode 277: Total Reward = 0.30
Episode 278: Total Reward = 0.50
Episode 279: Total Reward = 0.30
Episode 280: Total Reward = 0.10
Episode 281: Total Reward = 0.50
Episode 282: Total Reward = 0.50
Episode 283: Total Reward = 0.10
Episode 284: Total Reward = 0.50
Episode 285: Total Reward = 0.30
Episode 286: Total Reward = 0.10
Episode 287: Total Reward = 0.30
Episode 288: Total Reward = 0.10
Episode 289: Total Reward = 0.50
Episode 290: Total Reward = 0.40
Episode 291: Total Reward = 0.50
Episode 292: Total Reward = 0.50
Episode 293: Total Reward = 0.30

Episode 294: Total Reward = 0.50
Episode 295: Total Reward = 0.50
Episode 296: Total Reward = 0.10
Episode 297: Total Reward = 0.50
Episode 298: Total Reward = 0.50
Episode 299: Total Reward = 0.50
Episode 300: Total Reward = 0.10
Episode 301: Total Reward = 0.50
Episode 302: Total Reward = 0.50
Episode 303: Total Reward = 0.50
Episode 304: Total Reward = 0.50
Episode 305: Total Reward = 0.30
Episode 306: Total Reward = 0.30
Episode 307: Total Reward = 0.50
Episode 308: Total Reward = 0.50
Episode 309: Total Reward = 0.30
Episode 310: Total Reward = 0.50
Episode 311: Total Reward = 0.10
Episode 312: Total Reward = 0.50
Episode 313: Total Reward = 0.40
Episode 314: Total Reward = 0.50
Episode 315: Total Reward = 0.30
Episode 316: Total Reward = 0.50
Episode 317: Total Reward = 0.00
Episode 318: Total Reward = 0.50
Episode 319: Total Reward = 0.50
Episode 320: Total Reward = 0.50
Episode 321: Total Reward = 0.10
Episode 322: Total Reward = 0.50
Episode 323: Total Reward = 0.30
Episode 324: Total Reward = 0.50
Episode 325: Total Reward = 0.30
Episode 326: Total Reward = 0.50
Episode 327: Total Reward = 0.50
Episode 328: Total Reward = -0.10
Episode 329: Total Reward = 0.50
Episode 330: Total Reward = 0.50
Episode 331: Total Reward = 0.50
Episode 332: Total Reward = -0.10
Episode 333: Total Reward = 0.50
Episode 334: Total Reward = 0.20
Episode 335: Total Reward = 0.50
Episode 336: Total Reward = 0.50
Episode 337: Total Reward = -0.10
Episode 338: Total Reward = 0.50
Episode 339: Total Reward = 0.30
Episode 340: Total Reward = 0.50
Episode 341: Total Reward = 0.50
Episode 342: Total Reward = 0.50
Episode 343: Total Reward = 0.40
Episode 344: Total Reward = 0.10
Episode 345: Total Reward = 0.50
Episode 346: Total Reward = 0.00
Episode 347: Total Reward = 0.50
Episode 348: Total Reward = 0.50
Episode 349: Total Reward = -0.50
Episode 350: Total Reward = 0.10
Episode 351: Total Reward = 0.20
Episode 352: Total Reward = 0.00
Episode 353: Total Reward = 0.30
Episode 354: Total Reward = 0.30
Episode 355: Total Reward = 0.40
Episode 356: Total Reward = 0.50
Episode 357: Total Reward = 0.50
Episode 358: Total Reward = 0.10
Episode 359: Total Reward = 0.50
Episode 360: Total Reward = 0.30
Episode 361: Total Reward = 0.30
Episode 362: Total Reward = 0.50
Episode 363: Total Reward = 0.50
Episode 364: Total Reward = 0.50
Episode 365: Total Reward = 0.50
Episode 366: Total Reward = 0.50
Episode 367: Total Reward = 0.50
Episode 368: Total Reward = 0.50
Episode 369: Total Reward = 0.20
Episode 370: Total Reward = 0.50
Episode 371: Total Reward = 0.50
Episode 372: Total Reward = 0.30
Episode 373: Total Reward = 0.00
Episode 374: Total Reward = 0.50
Episode 375: Total Reward = 0.50
Episode 376: Total Reward = 0.50
Episode 377: Total Reward = 0.50
Episode 378: Total Reward = 0.20
Episode 379: Total Reward = 0.10
Episode 380: Total Reward = 0.50
Episode 381: Total Reward = 0.20
Episode 382: Total Reward = 0.50

Episode 383: Total Reward = 0.50
Episode 384: Total Reward = 0.30
Episode 385: Total Reward = -0.10
Episode 386: Total Reward = 0.30
Episode 387: Total Reward = -0.30
Episode 388: Total Reward = 0.30
Episode 389: Total Reward = 0.50
Episode 390: Total Reward = 0.10
Episode 391: Total Reward = 0.50
Episode 392: Total Reward = 0.50
Episode 393: Total Reward = 0.50
Episode 394: Total Reward = 0.50
Episode 395: Total Reward = 0.30
Episode 396: Total Reward = 0.40
Episode 397: Total Reward = 0.30
Episode 398: Total Reward = 0.50
Episode 399: Total Reward = 0.50
Episode 400: Total Reward = 0.10
Episode 401: Total Reward = 0.30
Episode 402: Total Reward = 0.30
Episode 403: Total Reward = 0.50
Episode 404: Total Reward = 0.30
Episode 405: Total Reward = 0.10
Episode 406: Total Reward = 0.30
Episode 407: Total Reward = 0.40
Episode 408: Total Reward = 0.50
Episode 409: Total Reward = 0.30
Episode 410: Total Reward = 0.40
Episode 411: Total Reward = 0.30
Episode 412: Total Reward = 0.40
Episode 413: Total Reward = 0.30
Episode 414: Total Reward = 0.30
Episode 415: Total Reward = 0.30
Episode 416: Total Reward = 0.00
Episode 417: Total Reward = 0.50
Episode 418: Total Reward = 0.30
Episode 419: Total Reward = 0.50
Episode 420: Total Reward = 0.50
Episode 421: Total Reward = 0.10
Episode 422: Total Reward = 0.00
Episode 423: Total Reward = 0.50
Episode 424: Total Reward = 0.50
Episode 425: Total Reward = 0.30
Episode 426: Total Reward = 0.10
Episode 427: Total Reward = 0.50
Episode 428: Total Reward = 0.30
Episode 429: Total Reward = 0.30
Episode 430: Total Reward = 0.30
Episode 431: Total Reward = 0.10
Episode 432: Total Reward = 0.50
Episode 433: Total Reward = 0.50
Episode 434: Total Reward = 0.30
Episode 435: Total Reward = 0.20
Episode 436: Total Reward = 0.50
Episode 437: Total Reward = 0.30
Episode 438: Total Reward = 0.50
Episode 439: Total Reward = 0.50
Episode 440: Total Reward = 0.50
Episode 441: Total Reward = 0.50
Episode 442: Total Reward = 0.30
Episode 443: Total Reward = 0.20
Episode 444: Total Reward = 0.50
Episode 445: Total Reward = 0.50
Episode 446: Total Reward = 0.30
Episode 447: Total Reward = 0.30
Episode 448: Total Reward = 0.20
Episode 449: Total Reward = 0.50
Episode 450: Total Reward = -0.10
Episode 451: Total Reward = 0.50
Episode 452: Total Reward = 0.50
Episode 453: Total Reward = -0.20
Episode 454: Total Reward = 0.30
Episode 455: Total Reward = 0.50
Episode 456: Total Reward = 0.30
Episode 457: Total Reward = 0.50
Episode 458: Total Reward = 0.30
Episode 459: Total Reward = 0.20
Episode 460: Total Reward = 0.50
Episode 461: Total Reward = 0.40
Episode 462: Total Reward = 0.10
Episode 463: Total Reward = 0.30
Episode 464: Total Reward = 0.40
Episode 465: Total Reward = 0.50
Episode 466: Total Reward = 0.50
Episode 467: Total Reward = 0.40
Episode 468: Total Reward = 0.40
Episode 469: Total Reward = 0.30
Episode 470: Total Reward = 0.50
Episode 471: Total Reward = 0.30

Episode 472: Total Reward = 0.40
Episode 473: Total Reward = 0.30
Episode 474: Total Reward = 0.50
Episode 475: Total Reward = 0.50
Episode 476: Total Reward = 0.50
Episode 477: Total Reward = 0.50
Episode 478: Total Reward = -0.20
Episode 479: Total Reward = 0.50
Episode 480: Total Reward = 0.40
Episode 481: Total Reward = 0.50
Episode 482: Total Reward = 0.50
Episode 483: Total Reward = 0.50
Episode 484: Total Reward = 0.50
Episode 485: Total Reward = 0.50
Episode 486: Total Reward = 0.20
Episode 487: Total Reward = 0.30
Episode 488: Total Reward = 0.50
Episode 489: Total Reward = 0.50
Episode 490: Total Reward = 0.40
Episode 491: Total Reward = 0.50
Episode 492: Total Reward = 0.50
Episode 493: Total Reward = 0.10
Episode 494: Total Reward = 0.40
Episode 495: Total Reward = 0.40
Episode 496: Total Reward = 0.20
Episode 497: Total Reward = 0.30
Episode 498: Total Reward = 0.40
Episode 499: Total Reward = 0.20
Episode 500: Total Reward = -0.50

Testing the agent...

Test Run: Path = [(0, 0), (0, 1), (1, 1), (2, 1), (2, 2), (2, 3), (3, 3)], Total Reward = 0.50

In []:

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js